



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO



Tema:

Arquitecturas cliente-servidor contemporáneas y diseño API-first

Docente:

Jenner Noel Che Méndez

Escuela:

Instituto Tecnológico de Chiná

Carrera:

Ingeniería en informática

Alumno:

Uriel Barrales Martínez

Semestre:

7 AI

Fecha:

28 de septiembre de 2026

Índice

1.1 Evolución del modelo cliente-servidor.....	3
1.2 Fundamentos de red aplicados.....	4
1.3 Arquitecturas de dos, tres y N capas	5
1.4 Estilos y protocolos	6
1.5 Principios de diseño y trade-offs del sistema	7
1.6 Estrategia API-first, diagramación C4 y ADRs	8
Referencias	9

1.1 Evolución del modelo cliente-servidor

El modelo cliente-servidor constituye el fundamento estructural de la informática distribuida contemporánea. En este paradigma, el proceso cliente actúa como la entidad de cómputo que desencadena activamente peticiones de servicio, mientras que el proceso servidor se ejecuta de manera persistente a la escucha de solicitudes para procesarlas y retornar una respuesta adecuada. La piedra angular de esta interacción es la noción de servicio, la cual abstrae y encapsula capacidades lógicas o de procesamiento de negocio detrás de una interfaz accesible y estandarizada, independizando por completo al consumidor de los detalles de implementación interna.

A medida que las aplicaciones ganaron en escala y heterogeneidad, la comunicación directa entre cliente y servidor puso de manifiesto acoplamientos rígidos y severas barreras de portabilidad. Para resolver estas deficiencias se introdujo el middleware, una capa de software de infraestructura posicionada estratégicamente entre el sistema operativo y las aplicaciones finales.

El middleware abstrae la complejidad subyacente de la red gestionando aspectos fundamentales como la serialización y deserialización de datos, el enrutamiento dinámico de mensajes, la autenticación y la coordinación de transacciones distribuidas a través de plataformas, redes y lenguajes de programación dispares.

La evolución continuó hacia los sistemas distribuidos modernos, definidos como conjuntos de nodos independientes interconectados por red que cooperan para ofrecer la percepción de una entidad única e integrada. Sin embargo, la ingeniería de estos sistemas exige superar las ocho falacias de la computación distribuida formuladas por L. Peter Deutsch y sus colegas en Sun Microsystems: asumir que la red es confiable, que la latencia es cero, que el ancho de banda es infinito, que la red es segura, que la topología no cambia, que hay un único administrador, que el costo de transporte es cero y que la red es totalmente homogénea. Mitigar estos supuestos erróneos requiere implementar patrones de resiliencia como disyuntores (circuit breakers), reintentos con desbordamiento exponencial (exponential backoff), mamparos (bulkheads) e interfaces estrictamente idempotentes.

1.2 Fundamentos de red aplicados

Las arquitecturas distribuidas dependen intrínsecamente de las capas del modelo TCP/IP. El proceso de localización comienza con el Domain Name System (DNS), el cual traduce nombres semánticos de dominio a direcciones de Protocolo de Internet (IP). La dirección IP ubica la interfaz física o virtual del nodo de destino, mientras que los puertos permiten multiplexar múltiples sockets dentro de un mismo host, direccionando el flujo de datos directamente hacia el proceso servidor correspondiente.

En la capa de transporte, el protocolo TCP asegura un canal orientado a conexión, ordenado y confiable mediante el apretón de manos de tres vías (three-way handshake: SYN, SYN-ACK, ACK), integrando control de flujo por ventana deslizante y control de congestión. No obstante, estas garantías introducen sobrecostos en tiempo de ida y vuelta (round-trip time) y pueden derivar en el problema de bloqueo de cabeza de línea (Head-of-Line Blocking), donde un paquete perdido detiene la entrega del buffer completo a la aplicación.

En la capa de aplicación, HTTP articula la interacción mediante un ciclo de petición y respuesta. El cliente envía una petición compuesta por un método semántico (GET, POST, PUT, DELETE, PATCH), una URI, encabezados de metadatos y un cuerpo opcional. El servidor procesa dicha solicitud y retorna una respuesta constituida por un código de estado numérico (1xx a 5xx), encabezados y la representación del recurso. Para asegurar el tránsito de la información, HTTPS integra la capa de seguridad TLS, la cual autentica al servidor (y opcionalmente al cliente mediante mTLS) mediante certificados X.509 y negocia claves criptográficas efímeras (como ECDHE), añadiendo intercambios iniciales que incrementan la latencia de conexión.

La latencia total experimentada por el cliente al interactuar con un servicio distribuido se descompone de forma aditiva en cuatro componentes físicos e infraestructurales: la latencia de propagación (tiempo de viaje de la señal por el medio físico a la velocidad de la luz en el medio), la latencia de transmisión (tiempo para inyectar los bits según el ancho de banda disponible), la latencia de procesamiento (cómputo requerido en routers, cortafuegos y servidores) y la latencia de cola (tiempo de espera en los buffers de memoria antes de ser atendido).

1.3 Arquitecturas de dos, tres y N capas

El diseño del lado del servidor ha evolucionado para optimizar la separación de responsabilidades y la capacidad de escalado. La arquitectura tradicional de dos capas conectaba directamente clientes pesados con servidores de bases de datos relacionales, acoplando la interfaz con la base de datos y complicando la seguridad y los despliegues. El modelo de tres capas introdujo un servidor de aplicaciones intermedio para aislar la lógica de dominio. Por su parte, el esquema de N capas dividió la estructura en múltiples niveles especializados (gateways de API, servicios de orquestación, cachés distribuidas y bases de datos analíticas) que escalan independientemente según su carga.

Respecto a la organización del código y el proceso de despliegue, el monolito modular concentra todo el sistema dentro de una única unidad ejecutable pero impone fronteras lógicas estrictas entre módulos a nivel de código. Esta estructura preserva llamadas en memoria de muy baja latencia, transacciones ACID nativas y simplicidad operativa. En contraposición, la arquitectura de microservicios descompone la aplicación en un conjunto de servicios autónomos orientados a dominios de negocio, donde cada servicio gestiona su propio almacenamiento de datos (Database-per-Service). Aunque otorga autonomía a los equipos y escalabilidad granular, incrementa sustancialmente la complejidad infraestructural y la gestión de la consistencia eventual.

Entre los paradigmas contemporáneos, la arquitectura orientada a eventos (EDA) utiliza intermediarios (event brokers) para emitir y consumir eventos asíncronos de forma desacoplada. El paradigma serverless o Function-as-a-Service (FaaS) abstrae por completo los servidores ejecutando código dentro de contenedores efímeros bajo demanda, tarifando únicamente por milisegundo ejecutado pero asumiendo latencias por arranque en frío (cold starts). Finalmente, Backend as a Service (BaaS) permite delegar módulos completos (autenticación, bases de datos o almacenamiento) a proveedores en la nube mediante SDKs, acelerando el desarrollo inicial a expensas de acoplamiento con el proveedor (vendor lock-in).

1.4 Estilos y protocolos

La elección de los protocolos condiciona el rendimiento y el acoplamiento del sistema. El estilo REST se centra en recursos identificados mediante URIs y manipulados con los métodos estándar de HTTP; su naturaleza sin estado (stateless) facilita el escalado horizontal y aprovecha la infraestructura de caché nativa de la web, posicionándolo como la opción estándar para APIs públicas. Por su parte, RPC y gRPC abstraen la invocación remota como ejecuciones de funciones locales. gRPC utiliza HTTP/2 como transporte y Protocol Buffers (Protobuf) como lenguaje de definición de interfaz y formato binario, ofreciendo esquemas fuertemente tipados, bajo tamaño de carga útil y streaming bidireccional, ideal para la comunicación interna entre microservicios de alto rendimiento.

Para interacciones en tiempo real, WebSocket establece un canal dúplex completo (full-duplex) y persistente sobre una única conexión TCP tras un apretón de manos inicial (HTTP Upgrade), reduciendo la sobrecarga de encabezados por mensaje. Como alternativa descendente, Server-Sent Events (SSE) ofrece un flujo unidireccional en tiempo real desde el servidor hacia el cliente utilizando la cabecera text/event-stream sobre HTTP estándar, integrando reconexión automática de forma nativa.

Los webhooks habilitan la comunicación asíncrona entre servidores mediante callbacks HTTP POST activados por eventos de dominio, eliminando el sondeo constante (polling) pero requiriendo firmas de autenticación (como HMAC) e idempotencia en el receptor. La mensajería asíncrona empleando intermediarios (brokers como RabbitMQ o logs distribuidos como Apache Kafka) desacopla temporalmente el envío de la recepción, absorbiendo picos de tráfico con garantías de entrega configurables. Los criterios de selección deben considerar si la interacción es sincrónica o asíncrona, la dirección del flujo de datos, la latencia tolerada, el volumen de datos y la capacidad de los clientes involucrados.

1.5 Principios de diseño y trade-offs del sistema

Construir arquitecturas distribuidas implica gestionar compromisos constantes entre atributos de calidad. Si bien aplicar una estricta separación de responsabilidades permite delimitar dominios de falla y escalar componentes individualmente, elevar la escalabilidad horizontal mediante la adición de nodos incrementa la complejidad técnica y los costos de infraestructura. Además, la distribución dificulta garantizar de forma simultánea una alta disponibilidad y una consistencia inmediata de los datos. La acumulación de decisiones apresuradas en estas dimensiones genera deuda operativa, exigiendo un esfuerzo desproporcionado para mantener la estabilidad en entornos de producción.

El soporte teórico de estos compromisos se formalizó inicialmente con el Teorema CAP de Eric Brewer, el cual postula que ante una partición de red (P), un sistema distribuido solo puede garantizar consistencia linealizable (C) o avance y disponibilidad (A), obligando a optar entre esquemas CP o AP. No obstante, el Teorema PACELC formulado por Daniel Abadi extiende CAP al incorporar la latencia cuando el sistema opera en condiciones normales sin fallos de red. PACELC establece que si hay una partición de red (P), se debe elegir entre Disponibilidad (A) o Consistencia (C); sino (Else - E), en operación normal, se debe elegir entre Latencia (L) o Consistencia (C).

PACELC clasifica los sistemas en cuatro categorías principales según sus prioridades operativas. Los sistemas PA/EL eligen Disponibilidad ante particiones y priorizan la baja Latencia en funcionamiento normal mediante replicación asíncrona, siendo comunes en bases de datos NoSQL como ScyllaDB, Apache Cassandra y Amazon DynamoDB. Los sistemas PC/EC priorizan la Consistencia en todo momento, pagando el costo de latencia al coordinar consensos o replicación síncrona, como en Google Spanner y CockroachDB. Los sistemas PA/EC garantizan Disponibilidad en partición, pero asumen latencia en normalidad para asegurar consistencia estricta. Finalmente, los sistemas PC/EL eligen Consistencia ante particiones, pero optimizan la Latencia en funcionamiento normal permitiendo lecturas desde réplicas secundarias.

1.6 Estrategia API-first, diagramación C4 y ADRs

La metodología API-first establece que las APIs deben diseñarse y acordarse como artefactos de primera clase antes de escribir código backend o desarrollar interfaces de usuario. Este enfoque transforma a la API en un contrato inicial estricto entre proveedores y consumidores, lo que permite generar servidores de simulación (mock servers), pruebas de contrato automatizadas y clientes tipados en paralelo, reduciendo los tiempos de integración y desacoplando los ciclos de desarrollo. Dentro de esta metodología destacan especificaciones estándar como OpenAPI 3.1 para interfaces HTTP sincrónicas y AsyncAPI 3.0 para arquitecturas asíncronas basadas en eventos.

Para la visualización de la arquitectura, el Modelo C4 creado por Simon Brown proporciona un marco jerárquico simplificado compuesto por cuatro niveles de abstracción. El Nivel 1 (Contexto del Sistema) muestra el sistema en su entorno, sus límites explícitos y sus relaciones con usuarios y otros sistemas externos. El Nivel 2 (Contenedores) descompone el sistema en sus unidades ejecutables e independientemente desplegables, como aplicaciones web, servicios API y bases de datos, especificando tecnologías y protocolos. El Nivel 3 (Componentes) detalla los módulos o bloques de código que estructuran internamente a un contenedor específico. El Nivel 4 (Código) proporciona una representación orientada a clases o entidad-relación, habitualmente generada de forma automática.

Por último, la gobernanza de las decisiones técnicas se gestiona mediante los Registros de Decisión Arquitectónica (ADR). Un ADR es un documento formal de texto plano que captura una decisión estructural relevante junto con su contexto e impactos. Utilizando formatos consolidados como la plantilla de Michael Nygard o el esquema MADR, todo ADR debe detallar su Título secuencial, Estatus (Propuesto, Aprobado, Deprecado), Contexto técnico e impulsores de negocio, la Decisión adoptada y las Consecuencias, analizando de manera transparente tanto los beneficios obtenidos como las complejidades y compromisos asumidos.

Referencias

- Abadi, D. J. (2012). Consistency tradeoffs in modern distributed database system design. *Computer*, 45(2), 37–42. <https://doi.org/10.1109/MC.2012.33>
- AsyncAPI Initiative. (2024, 12 de marzo). AsyncAPI 3.0 specification cheat sheet. Bump.sh. <https://bump.sh/blog/asyncapi-30-cheatsheet/>
- Brown, S. (2023). C4 model | Lightweight standard for visualizing software architecture. C4Model.info. <https://c4model.info/>
- Nygard, M. (2023). Architecture decision record (ADR). GitHub Repository. <https://github.com/architecture-decision-record/architecture-decision-record>
- OpenAPISpec. (2026, 12 de agosto). What is AsyncAPI and how does it differ from OpenAPI? OpenAPISpec Reference. <https://openapispec.com/docs/what/what-is-asyncapi-and-how-does-it-differ-from-openapi/>